

DISCIPLINA DE POO EM JAVA

Aula 3: Introdução à Orientação a Objetos

Bem-vindo à terceira aula da disciplina de **Programação Orientada a Objetos em Java**. Hoje vamos explorar os fundamentos do paradigma orientado a objetos — a base conceitual que sustenta toda a programação Java moderna.



Objetivos de Aprendizagem

Ao final desta aula, você será capaz de compreender e aplicar os conceitos fundamentais da orientação a objetos em seus primeiros projetos Java.

1

Compreender o Paradigma

Entender o que é orientação a objetos, sua origem e por que ela é amplamente adotada na indústria.

2

Dominar os Conceitos Base

Distinguir classe de objeto, compreender instanciação e os quatro pilares: abstração, encapsulamento, herança e polimorfismo.

3

Pensar Orientado a Objetos

Modelar problemas do mundo real identificando objetos, responsabilidades e colaborações.

4

Aplicar em Java

Reconhecer como Java implementa o paradigma OO e as convenções de nomenclatura adotadas.



O que é Orientação a Objetos?

A **Orientação a Objetos (OO)** é uma forma de programar em que organizamos o sistema em entidades. Cada entidade guarda informações e também sabe realizar ações.

Em vez de escrever uma sequência de instruções, você modela o mundo real em termos de entidades que interagem entre si.

Paradigma Procedural

O programa é visto mais como uma sequência de passos e funções;

Paradigma O.O.

O programa é visto como um conjunto de objetos que interagem.

Ou seja, o programa não é pensado apenas como vários comandos espalhados. Ele é organizado em partes, e cada parte tem sua função, suas informações e suas ações, atuando junto das outras para fazer o sistema funcionar.



O que é Orientação a Objetos?

Analogia do Mundo Real

Um carro possui características, como cor e velocidade, e também realiza ações, como acelerar e frear.

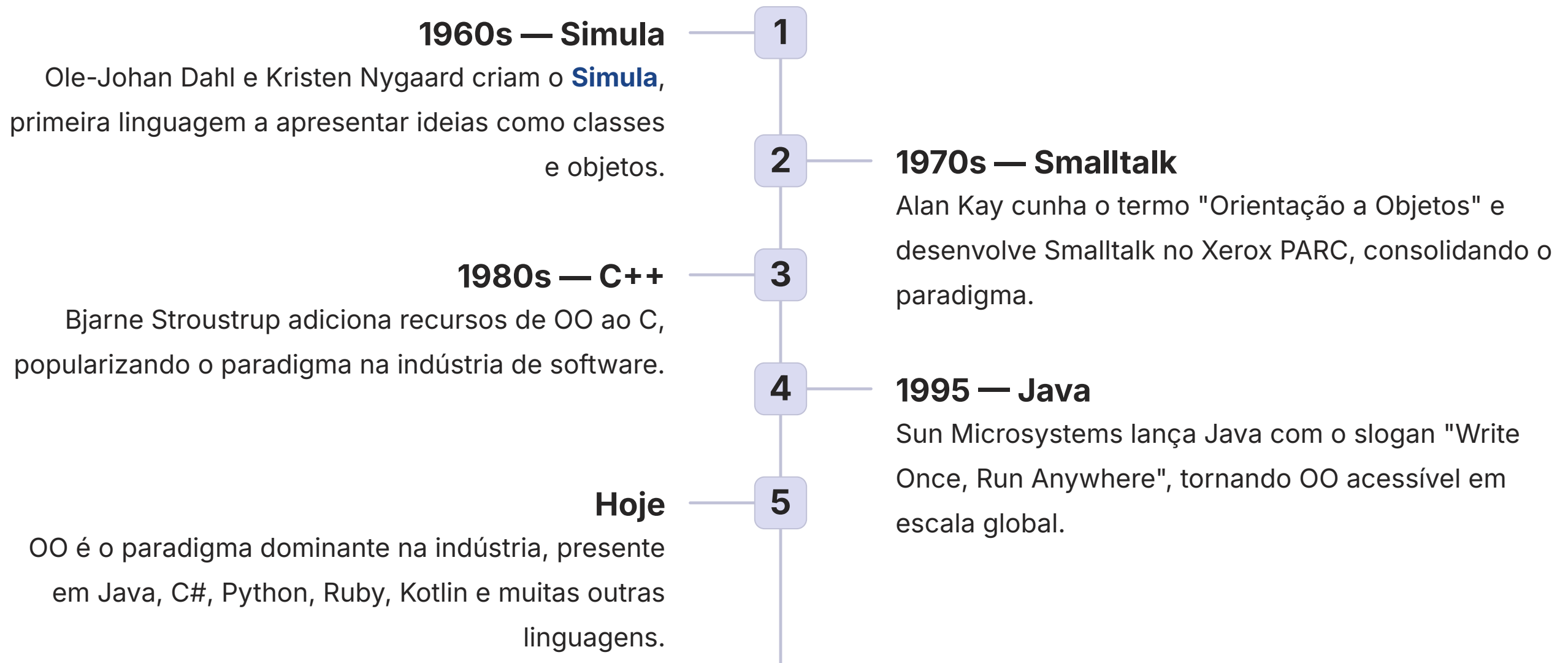
Em Orientação a Objetos, pensamos de forma semelhante: um objeto reúne dados e comportamentos em uma mesma estrutura.

Por que usar?

A Orientação a Objetos facilita a representação de problemas do mundo real, ajuda a organizar melhor o código, favorece a reutilização de componentes e torna o software mais simples de entender, manter e expandir.

Origem e Evolução Histórica

A orientação a objetos não surgiu do nada, foi fruto de décadas de evolução no pensamento sobre como organizar programas complexos.



Paradigma Procedural vs. Orientado a Objetos

Antes de estudar a Orientação a Objetos, é importante entender como ela se diferencia da programação procedural, que você provavelmente já conhece da linguagem C.

Programação

Procedural

Foco em **funções e procedimentos** (Etapas do Programa)

- Dados e Comportamentos (ações) ficam separados;
- O programa costuma ser organizado como uma sequência de passos. Seguindo um fluxo de execução linear;
- Funciona bem em programas menores e mais diretos;
- Pode ficar mais difícil de organizar à medida que o sistema cresce;
 - Exemplos: C, Pascal, COBOL

Programação Orientada a Objetos

Foco em **objetos e suas interações**

- Os dados e ações ficam reunidos no mesmo elemento;
- O programa é organizado em torno de objetos;
- Representa melhor os elementos do problema;
- Facilita a organização em sistemas maiores;
 - Exemplos: Java, C#, Python, Kotlin.



Importante: Nem sempre a Orientação a Objetos será a melhor escolha. Em programas mais simples, scripts ou certos problemas específicos, outras abordagens também podem ser mais adequadas.

Conceito de Objeto

Logo um **objeto** é a unidade fundamental da OO. Ele representa uma entidade do mundo real que pode ser entendido por três aspectos principais:

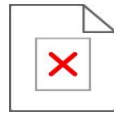


Estado

São os **atributos** (ou campos) que descrevem as características do objeto em um determinado momento.

Exemplo: um carro tem *cor = "vermelho"*, *velocidade = 0*, *combustivel = 50*.

É o que o objeto **tem**.

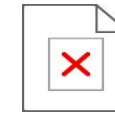


Comportamento

Os **métodos** que definem o que o objeto pode fazer.

Exemplo: o carro pode *acelerar()*, *frear()*, *ligar()*. São as ações que o objeto executa.

É o que o objeto **faz**.



Identidade

Cada objeto é **único**, mesmo que dois objetos tenham os mesmos atributos. Dois carros vermelhos são dois objetos distintos na memória do sistema.

Exemplo: CPF, Matrícula, ID, etc.

É o que torna o objeto **único** no sistema.

 **Analogia:** Podemos pensar em uma pessoa como um objeto:

Estado: nome, idade, altura; **Comportamento:** falar, caminhar, estudar; **Identidade:** o que a diferencia das demais.

Conceito de Classe

Se o objeto é algo criado no sistema (uma entidade concreta), a **classe** é o molde, modelo ou blueprint que define como os objetos serão criados. A classe especifica quais atributos e métodos cada objeto terá.

O que é uma Classe?

Uma classe é o modelo que define como os objetos de um mesmo tipo serão criados.

Ela descreve quais características e quais ações esses objetos terão.

Então uma classe:

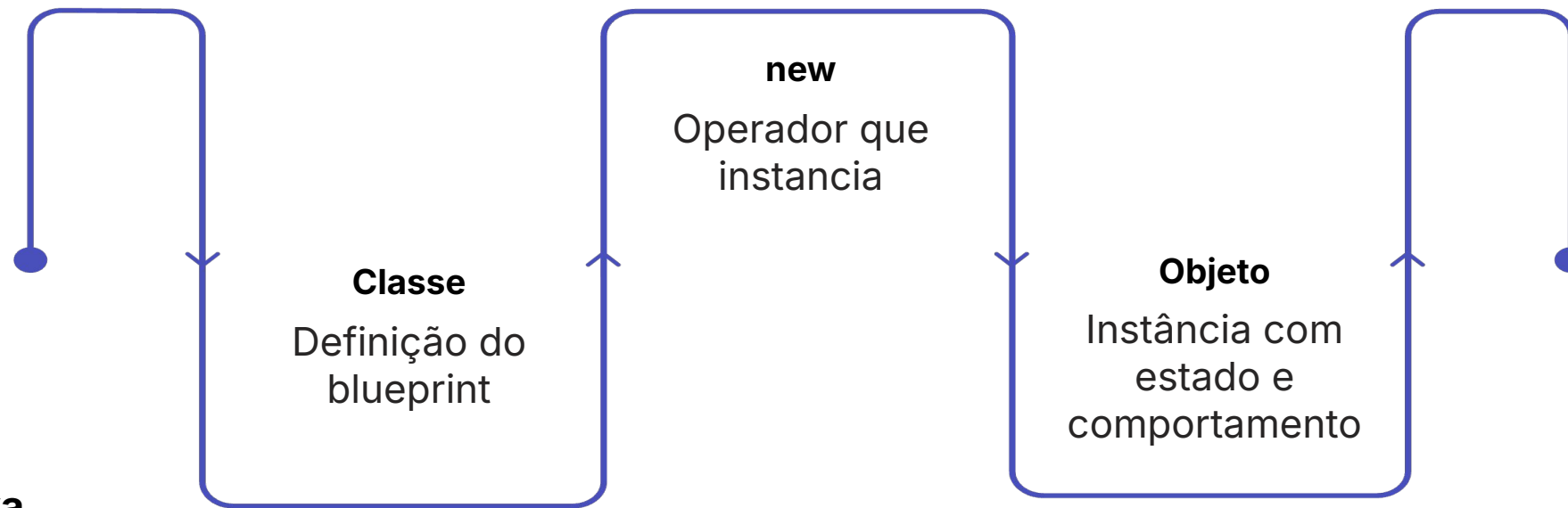
- Define a estrutura dos objetos;
- Especifica os atributos;
- Declara os métodos;
- Serve como base para criar objetos no programa.

Exemplo: Classe Carro

```
public class Carro {  
    // Atributos (estado)  
    String cor;  
    String modelo;  
    int velocidade;  
  
    // Métodos (comportamento)  
    void acelerar(int valor) {  
        velocidade += valor;  
    }  
  
    void frear(int valor) {  
        velocidade -= valor;  
    }  
}
```


Relação Classe e Objeto: Instanciação

O processo de criar um objeto a partir de uma classe é chamado de **instanciação**. Em Java, usamos a palavra-chave `new` para instanciar uma classe e obter um objeto na memória.



Criando Objetos em Java

```
// Criando dois objetos da classe Carro
Carro meuCarro = new Carro();
Carro carroDaMaria = new Carro();

// Definindo a cor de cada carro
meuCarro.cor = "vermelho";
carroDaMaria.cor = "azul";

// Fazendo o meu carro acelerar
meuCarro.acelerar(50);
```

Conceitos-chave

Classe: o molde (existe uma vez)

Objeto: a instância (podem existir muitos)

new: operador de instanciação em Java

Referência: variável que aponta para o objeto na memória heap

Cada instância mantém seu **próprio estado** independente

Os 4 Pilares da Orientação a Objetos

A OO se sustenta em quatro princípios fundamentais. Nesta aula fazemos uma visão geral de cada pilar — eles serão aprofundados nas próximas aulas.



Abstração

Focar no que importa.

Ex: um carro tem modelo e velocidade, mas ignoramos a composição química do metal.



Encapsulamento

Proteger os dados internos do objeto, expondo apenas o necessário. Usamos modificadores de acesso (`private`, `public`) para controlar o acesso aos atributos e métodos.



Herança

Permite que uma classe **herde** atributos e métodos de outra. Promove reutilização de código. Em Java: `class Esportivo extends Carro`. O filho herda do pai.



Polimorfismo

A capacidade de um objeto assumir **diferentes formas** dependendo do objeto que o executa. Confere enorme flexibilidade ao sistema.

Pensamento Orientado a Objetos

Antes de escrever qualquer código, o programador OO precisa **modelar o problema**. Isso significa observar o domínio do sistema e identificar quais entidades existem, o que elas sabem e o que elas fazem.

1 Identifique os Objetos

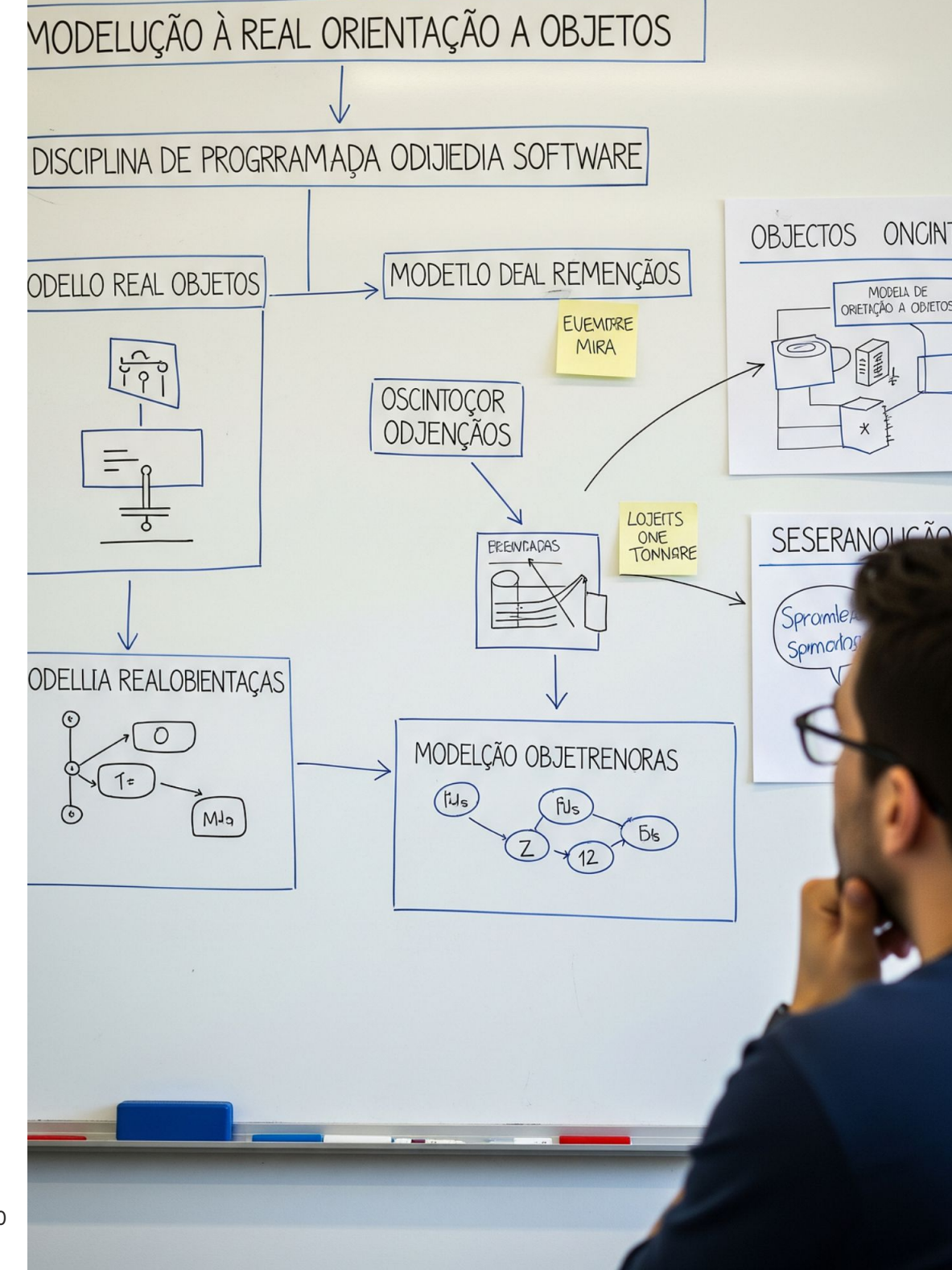
Observe os **substantivos** no problema. "Um aluno realiza uma matrícula em uma disciplina" → objetos: *Aluno*, *Matrícula*, *Disciplina*.

2 Defina Responsabilidades

Cada objeto deve ter responsabilidades claras. O que ele **conhece** (atributos) e o que ele **faz** (métodos)? Responsabilidade única por objeto.

3 Modele Colaborações

Objetos **colaboram** entre si trocando mensagens (chamando métodos). Um *Pedido* usa um *Produto*, que consulta um *Estoque*.



Exemplo Prático: Modelando uma Conta Bancária

Vamos aplicar o pensamento OO em um exemplo clássico: um sistema bancário simplificado. Identificamos as classes, seus atributos e métodos.

Identificando o Modelo

Domínio: Sistema de banco simples com clientes e contas.

Classes identificadas: *Cliente*, *Conta*, *Transacao*

Atributos de Conta: número, saldo, titular

Métodos de Conta: depositar(), sacar(), consultarSaldo()

```
public class Conta {  
    private String numero;  
    private double saldo;  
    private String titular;  
  
    public void depositar(double valor) {  
        saldo += valor;  
    }  
  
    public boolean sacar(double valor) {  
        if (saldo >= valor) {  
            saldo -= valor;  
            return true;  
        }  
        return false;  
    }  
  
    public double consultarSaldo() {  
        return saldo;  
    }  
}
```


Exemplo Prático: Modelando uma Conta Bancária

Vamos aplicar o pensamento OO em um exemplo clássico: um sistema bancário simplificado. Identificamos as classes, seus atributos e métodos.

Identificando o Modelo

Domínio: Sistema de banco simples com clientes e contas.

Classes identificadas: *Cliente, Conta, Transacao*

Atributos de Conta: **número, saldo, titular**

Métodos de Conta: depositar(), sacar(), consultarSaldo()

```
public class Conta {  
    private String numero;  
    private double saldo;  
    private String titular;  
  
    public void depositar(double valor) {  
        saldo += valor;  
    }  
  
    public boolean sacar(double valor) {  
        if (saldo >= valor) {  
            saldo -= valor;  
            return true;  
        }  
        return false;  
    }  
  
    public double consultarSaldo() {  
        return saldo;  
    }  
}
```

Exemplo Prático: Modelando uma Conta Bancária

Vamos aplicar o pensamento OO em um exemplo clássico: um sistema bancário simplificado. Identificamos as classes, seus atributos e métodos.

Identificando o Modelo

Domínio: Sistema de banco simples com clientes e contas.

Classes identificadas: *Cliente*, *Conta*, *Transacao*

Atributos de Conta: número, saldo, titular

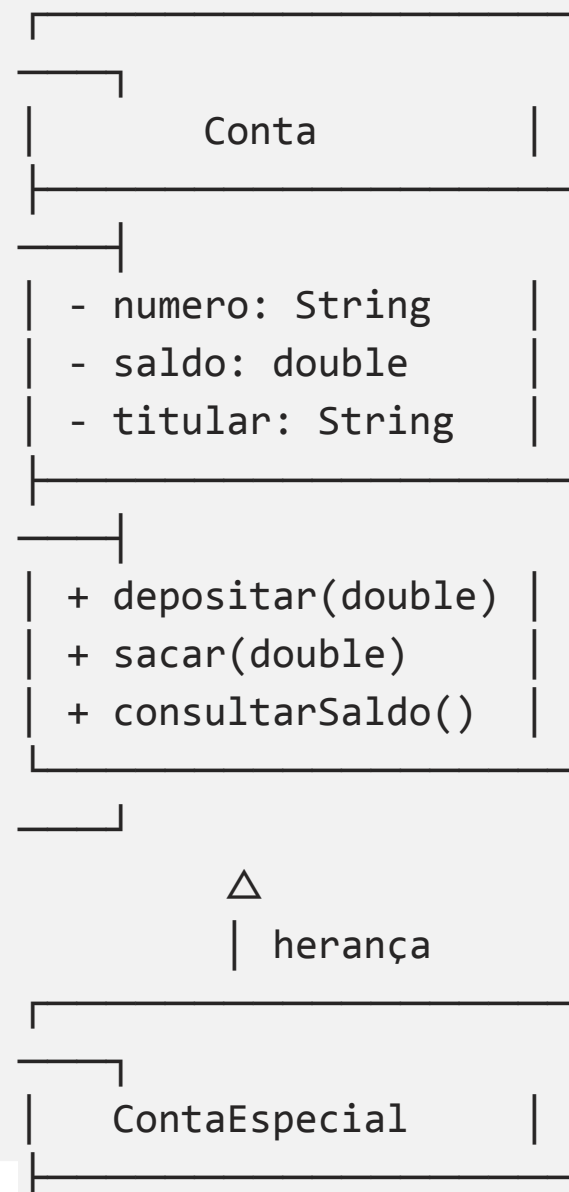
Métodos de Conta: **depositar()**, **sacar()**, **consultarSaldo()**

```
public class Conta {  
    private String numero;  
    private double saldo;  
    private String titular;  
  
    public void depositar(double valor) {  
        saldo += valor;  
    }  
  
    public boolean sacar(double valor) {  
        if (saldo >= valor) {  
            saldo -= valor;  
            return true;  
        }  
        return false;  
    }  
  
    public double consultarSaldo() {  
        return saldo;  
    }  
}
```

Exemplo Prático: Modelando uma Conta Bancária

Vamos aplicar o pensamento OO em um exemplo clássico: um sistema bancário simplificado. Identificamos as classes, seus atributos e métodos.

Diagrama UML Básico



O diagrama UML representa visualmente a estrutura das classes e seus relacionamentos, ferramenta essencial na modelagem OO.

Vantagens da Orientação a Objetos

A adoção da OO traz benefícios concretos e mensuráveis no desenvolvimento de software profissional, especialmente em projetos de médio e grande porte.

Reutilização de Código

Classes e hierarquias de herança permitem aproveitar código existente em novos contextos, reduzindo duplicação e esforço de desenvolvimento.

Manutenibilidade

O encapsulamento isola mudanças: alterar a implementação interna de uma classe não afeta quem a usa, desde que a interface pública seja mantida.

Extensibilidade

Novas funcionalidades são adicionadas criando subclasses ou novas classes, sem modificar o código existente — respeitando o princípio Open/Closed.

Modularidade e Trabalho em Equipe

Classes bem definidas podem ser desenvolvidas por diferentes membros da equipe em paralelo, acelerando projetos corporativos e facilitando a colaboração.

OO no Java

Java foi projetado desde o início como uma linguagem orientada a objetos. Quase tudo em Java é um objeto — com poucas exceções como os tipos primitivos (`int`, `double`, `boolean`).

Tudo é (quase) Objeto

Em Java, toda classe herda implicitamente de `Object` — a superclasse raiz de todas as classes. Tipos primitivos têm wrappers: `Integer`, `Double`, `Boolean`.

Pacotes e Organização

Classes são organizadas em **pacotes** (package), que funcionam como namespaces. Ex: `br.edu.universidade.aluno`. Evitam conflitos de nomes e organizam o projeto.

Convenções de Nomenclatura

Classes: `PascalCase` (ex: `ContaBancaria`). Métodos e atributos: `camelCase` (ex: `consultarSaldo`). Constantes: `UPPER_SNAKE_CASE`. Pacotes: tudo minúsculo.



Exercícios de Fixação

Pratique os conceitos desta aula com os exercícios abaixo. Tente resolver antes de consultar qualquer material — o esforço de recuperação é a melhor forma de consolidar o aprendizado.

1

Identifique Objetos

Em um sistema de biblioteca, identifique pelo menos **4 classes**. Para cada uma, liste 3 atributos e 2 métodos. Ex: Livro, Leitor, Empréstimo...

2

Modelo de Carro

Crie a classe `Carro` em Java com atributos *modelo*, *cor* e *velocidade*. Implemente os métodos `acelerar()`, `frear()` e `exibirInfo()`. Instancie 2 objetos no `main()`.

3

Reflexão Conceitual

Explique com suas palavras a diferença entre **classe** e **objeto** usando uma analogia diferente das apresentadas em aula. Apresente também um exemplo do pilar de encapsulamento.

4

Desafio: Diagrama UML

Desenhe um diagrama UML básico para um sistema de **escola** com as classes *Aluno*, *Professor* e *Disciplina*. Inclua atributos, métodos e relacionamentos entre as classes.

Próximos Passos

Esta aula estabeleceu a base conceitual da orientação a objetos. Nas próximas aulas, mergulharemos em cada tópico com muito mais profundidade e código Java.

01

Aula 4: Classes e Objetos em Profundidade

Construtores, sobrecarga, `this`, atributos de instância vs. classe (`static`), inicialização de objetos e gerenciamento de memória no Java.

02

Aula 5: Encapsulamento na Prática

Modificadores de acesso (`private`, `protected`, `public`), métodos *getters* e *setters*, JavaBeans e boas práticas de encapsulamento em projetos reais.

03

Aula 6: Herança e Polimorfismo

Hierarquias de classes, `extends`, sobrescrita de métodos (`@Override`), classe `Object`, polimorfismo de subtipo e classes abstratas.

  **Para esta semana:** Complete os exercícios de fixação, revise o material desta aula e leia o Capítulo 3 do livro-texto sobre Classes e Objetos em Java. Dúvidas? Traga para o fórum ou para o início da próxima aula!